Dictionary

A **dictionary** is a collection that stores data in **key-value pairs**.

Important Points

- Dictionary elements are stored as **key-value pairs**.
- Dictionaries are enclosed in { } curly braces.
- Keys should be unique.
- The notes describe dictionary keys as strings.
- A dictionary can store multiple data types.
- Dictionaries are ordered and changeable.
- Duplicate keys are not allowed.

Example

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}

print(thisdict)
```

---

2. Dictionary Characteristics

Ordered

Dictionary items maintain their order.

Changeable

Dictionary values can be changed after the dictionary is created.

Example:

```
thisdict["year"] = 1965
```

No Duplicate Keys

Dictionary keys should not be duplicated. If the same key is assigned again, the previous value is replaced.

---

3. Dictionary Length

The len() function returns the number of items in a dictionary.

Example

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}

print(len(thisdict))
```

Output

3

---

4. Accessing Dictionary Items

Dictionary values can be accessed using their keys.

Using Square Brackets

```
thisdict = {
```

```
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}

x = thisdict["model"]

print(x)
```

Output

Mustang

Using get()

The get() method can also be used to access a value.

```
x = thisdict.get("model")

print(x)
```

Output

Mustang

---

5. keys()

The keys() method returns a list-like view containing the keys of the dictionary.

Example

```
x = thisdict.keys()

print(x)
```

The keys are:
brand
model
year

---

6. values()

The values() method returns a view containing all the values in the dictionary.

Example

```
x = thisdict.values()

print(x)
```

Example values:
Ford
Mustang
1964

---

7. items()

The items() method returns each dictionary item as a tuple containing the key and value.

Example

```
x = thisdict.items()

print(x)
```

Conceptually, the result contains:
("brand", "Ford")

("model", "Mustang")
("year", 1964)

---

8. Updating a Dictionary
The update() method updates the dictionary with items from the given argument.
Example

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}

thisdict.update({"year": 2020})

print(thisdict)
```

The value of "year" is updated.

---

9. Adding Items to a Dictionary
A new key-value pair can be added by assigning a value to a new key.
Example

```
thisdict["color"] = "Red"

print(thisdict)
```

The dictionary now contains:
brand → Ford
model → Mustang
year → 1964
color → Red

---

10. Removing Items from a Dictionary
There are several methods for removing items from a dictionary.
Using pop()
The pop() method removes the item with the specified key.

```
thisdict.pop("model")

print(thisdict)
```

Using popitem()
The popitem() method removes the last inserted item.

```
thisdict.popitem()

print(thisdict)
```

Using del
The del statement can remove a specific item.

```
del thisdict["model"]
```

Deleting the Entire Dictionary

del thisdict
This deletes the dictionary completely.

---

11. clear()
The clear() method empties the dictionary.
Example
thisdict.clear()

print(thisdict)
Output
{}

---

12. Looping Through a Dictionary
A dictionary can be traversed using a for loop.
Loop Through Keys
for x in thisdict:
  print(x)
This prints the dictionary keys.
Loop Through Values
for x in thisdict:
  print(thisdict[x])
This prints the values.

Tuple

1. What is a Tuple?
A **tuple** is an ordered collection of elements.
The main difference between a **list** and a **tuple** is that a tuple is **immutable**.
Immutable means that once a tuple is created, its elements cannot be changed.
Example
tup1 = (2, "Hello", "Python")

print(tup1)
Output:
(2, 'Hello', 'Python')

---

2. Characteristics of Tuple
A tuple has the following characteristics:

- **Ordered** – Elements maintain their position.
- **Immutable** – Elements cannot be changed.
- **Allows duplicates** – The same value can appear multiple times.
- **Supports indexing**.
- **Supports slicing**.
- Can contain different data types.

Example:
student = ("Abhinav", 24, 85.5, True)

print(student)

---

3. Creating a Tuple

Normal Tuple

t1 = (1, 2, 3, 4)

print(t1)

Tuple with Different Data Types

t1 = (10, "Python", 3.14, True)

print(t1)

Single-Element Tuple

A comma is required for a single-element tuple.

t1 = (10,)

print(type(t1))

Without the comma:

t1 = (10)

print(type(t1))

This is an integer, not a tuple.

---

4. Accessing Tuple Elements

Tuple elements are accessed using indexes.

The index starts from 0.

lang = ("Python", "C", "C++")

print(lang[0])

print(lang[2])

Output:

Python

C++

Index Positions

Python C C++

0 1 2

---

5. Negative Indexing

Negative indexes start from the end.

lang = ("Python", "C", "C++")

print(lang[-1])

print(lang[-2])

Output:

C++

C

---

6. Tuple Cannot Be Modified

Tuples are immutable.

Therefore, this produces an error:

```
lang = ("Python", "C", "C++")

lang[2] = "Java"
```
You cannot directly change an existing tuple element.

---

7. Looping Through a Tuple

A for loop can be used to access every element.

```
fruits = ("apple", "banana", "orange")

for fruit in fruits:
   print(fruit)
```
Output:
apple
banana
orange

---

8. Tuple Methods

Python tuples have **two main built-in methods**:

1. count()
2. index()

---

9. count()

The count() method returns the number of times a specified value occurs in a tuple.

Syntax

```
tuple.count(value)
```
Example

```
vowels = ("a", "e", "i", "o", "i", "u")

print(vowels.count("i"))
```
Output:
2
Because "i" occurs two times.

---

10. index()

The index() method returns the position of the **first occurrence** of a specified value.

Example

```
vowels = ("a", "e", "i", "o", "i", "u")

print(vowels.index("e"))
```
Output:
1
The first "e" is at index 1.

---

11. Tuple Operations

Apart from tuple methods, Python provides several operations and built-in functions that can be used with tuples.

11.1 max()

Returns the largest value.
numbers = (2, 3, 4, 5, 7, 8, 9)

print(max(numbers))
Output:
9

11.2 min()
Returns the smallest value.
numbers = (2, 3, 4, 5, 7, 8, 9)

print(min(numbers))
Output:
2

11.3 sum()
Returns the sum of numeric elements.
numbers = (2, 10, 8, 3, 7)

print(sum(numbers))
Output:
30

11.4 len()
Returns the number of elements.
numbers = (10, 20, 30, 40)

print(len(numbers))
Output:
4

---

12. Converting Tuple to List
The list() function converts a tuple into a list.
numbers = (1, 22, 2, 232.33)

print(list(numbers))
Output:
[1, 22, 2, 232.33]
This is useful when you need to modify the data.

---

13. Tuple Concatenation
Two tuples can be joined using the + operator.
t1 = (1, 2, 3)
t2 = (4, 5, 6)

print(t1 + t2)
Output:
(1, 2, 3, 4, 5, 6)

---

14. Adding Corresponding Tuple Elements

The zip() function can be used to combine corresponding elements.

```
t1 = (1, 2, 3)
t2 = (4, 5, 6)

new = []

for i, j in zip(t1, t2):
   new.append(i + j)

print(tuple(new))
```

Output:
(5, 7, 9)
Explanation:
1 + 4 = 5
2 + 5 = 7
3 + 6 = 9

---

15. Tuple Repetition

The * operator can repeat a tuple multiple times.

```
d = (1, 2, 3)

print(d * 3)
```

Output:
(1, 2, 3, 1, 2, 3, 1, 2, 3)

---

16. Membership Operation

The in and not in operators check whether an element exists in a tuple.

```
d = (2, 3, 4, 5, 6, 6, 7, 7, 87, 88, 8)

print(22 in d)
print(22 not in d)
```

Output:
False
True

---

17. Identity Operation

The is and is not operators can check whether two variables refer to the same object.
Example:

```
t1 = (1, 2, 3)
t2 = (1, 2, 3, 5, 35, 34)

print(t1 is not t2)
```

Output:
True

---

18. Iterating Through a Tuple

We can use a for loop to access each element.

```
d = (1, 2, 3, 35, 4, 6, 7, 8)

for i in d:
    print(i)
```

Output:

```
1
2
3
35
4
6
7
8
```

---

19. Nested Tuple

A **nested tuple** is a tuple containing other tuples.

```
nested = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 9)
)
```

---

20. Accessing Nested Tuple

Use multiple indexes.

```
nested = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 9)
)

print(nested[0][0])
```

Output:

```
1
```

Explanation:

```
nested[0]      → (1, 2, 3)

nested[0][0]   → 1
```

---

21. Searching in a Nested Tuple

We can search for a number using nested loops.

```
matrix = (
    (1, 2, 3),
    (4, 5, 6),
    (2, 8, 9)
)

num = 7
```

```
exists = False

for i in matrix:
    for j in i:
        if j == num:
            exists = True

if exists:
    print("True")
else:
    print("False")
```

Output:

False

Because 7 is not present.